IIITDM KURNOOL · DBMS PROJECT

BusBook
Team User Manual

Run the app, share it with teammates, book seats, manage waiting lists, and show the database features.

Start here

Already installed on the leader's laptop? Go to page 3 for the one-line launcher. New computer? Use page 2 first. This manual describes a local classroom demonstration; payments and refunds are simulated.

Choose your setup

Mode	Who can open it	Internet needed while running?
One laptop	Only the host laptop at 127.0.0.1	No
Shared local network	Teammates on the same Wi-Fi or hotspot	No; the network may also have internet
Public internet	Anyone from anywhere	Not configured in this project

How the pieces connect

Browser → Django app → Oracle XE

The browser shows routes and tickets. Django handles sign-in and booking actions. Oracle stores buses, seats, trips, bookings, payments, queue entries, audit events, and reports. GitHub stores the source code, not the live database.

Repository: github.com/karthik-goud227/bus-ticket-booking-system

01 / FRESH COMPUTER

Install your own copy

Skip this page if you are only opening the app from the leader's laptop. A separate installation has its own Oracle data and its own users.

Required on a Windows host	Purpose
Oracle Database 21c XE; xepdb1 service	Stores all project tables and data
Python 3.14 and a browser	Runs the Django application
Git or a downloaded ZIP	Gets the source code
SQL Developer (recommended)	Views Oracle tables, triggers, views and queries

1. Get the repository

```
git clone https://github.com/karthik-goud227/bus-ticket-booking-system.git
cd bus-ticket-booking-system
```

You can instead download and extract the ZIP from GitHub. Open PowerShell in the extracted project folder.

2. Create the Oracle project account

In SQL Developer, connect as an Oracle administrator to service xepdb1. Choose your own private password, then run:

```
CREATE USER BUSBOOKING IDENTIFIED BY "ChooseYourOwnPassword";
GRANT CREATE SESSION, CREATE TABLE, CREATE SEQUENCE,
      CREATE VIEW, CREATE TRIGGER, CREATE PROCEDURE TO BUSBOOKING;
ALTER USER BUSBOOKING QUOTA UNLIMITED ON USERS;
```

Check that BUSBOOKING can connect to localhost:1521/xepdb1. Do not recreate the account if it already exists.

3. Prepare Python and tables

```
py -3.14 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
$secure = Read-Host "BUSBOOKING password" -AsSecureString
$env:DB_PASSWORD = [System.Net.NetworkCredential]::new("", $secure).Password
.\.venv\Scripts\python.exe manage.py migrate
.\.venv\Scripts\python.exe manage.py createsuperuser
```

The last password is for the website admin account; it is separate from the Oracle password. Migrations create the schema rules and reports automatically. Never commit real passwords or database dumps to GitHub.

02 / START AND SHARE

Offline, online Wi-Fi, or both

'Offline' means the booking app does not need internet. For teammates to connect, the devices still need a shared local Wi-Fi or hotspot network.

Only your laptop

With Oracle running and DB_PASSWORD set in this PowerShell window:

```
.\.venv\Scripts\python.exe manage.py runserver
# Open http://127.0.0.1:8000/ on the host laptop
```

Your team on college Wi-Fi or a phone hotspot

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\start_team.ps1
```

The launcher asks for the BUSBOOKING password, starts the server, and prints the current teammate link (for example http://192.168.1.10:8000/). Your teammates join the same network and open that link. Keep the PowerShell window and host laptop running.

Switching networks

Stop the server with Ctrl+C. Connect all devices to the new Wi-Fi or hotspot. Run the launcher again and share its new link. The host IP may change; the accounts and bookings stay in Oracle on the laptop.

Situation	What to expect
College Wi-Fi with internet	Local app works; GitHub pushes can work too. Some campus networks block devices from reaching each other.
Hotspot without internet	Local app works for connected devices. GitHub pushes wait until internet returns.
Internet is lost mid-demo	Bookings continue if the local network and Oracle stay available.

Public access is not enabled

The GitHub repository does not host the live app. This project uses a development server with local HTTP settings; do not expose it to the public internet or forward a router port. Public hosting needs a separate production deployment.

03 / ACCOUNTS AND FLEET

Prepare people and trips

Passenger accounts

- Each teammate opens Sign up (or /signup/) and chooses a personal username and password. These are website accounts, not Oracle or GitHub accounts.
- A passenger can search trips, reserve seats, see their own bookings, join a waiting list, and accept an offer.

Admin accounts

- The leader signs into /admin/ with the Django superuser created during setup.
- For a separate teammate admin account: let the teammate sign up with another username. In /admin/ → Users, select it, enable Staff status and Superuser status, then save. Only do this for trusted teammates; they can change all project data.

Build a fleet

- In /admin/, add a bus with its registration, operator, bus type, and physical seats. Then add a future trip with origin, destination, departure, arrival, fare, and that bus.
- A trip gets separate seat inventory. The same physical bus can have multiple trips, but active trips for that bus cannot overlap. Different buses can run at the same time.

Quick demo data

```
.\.venv\Scripts\python.exe manage.py seed_fleet
```

Run this from a PowerShell window with DB_PASSWORD set. The command defines 12 demo buses and 32 future trips across Andhra Pradesh, Telangana, Karnataka, and Tamil Nadu. It keeps existing bookings. Repeating it on the same date does not duplicate those specific demo trips; running it on a later date may add a new future schedule.

Two kinds of accounts

Oracle BUSBOOKING is the database login used by Django. A Django passenger or admin account signs into the website. GitHub credentials are only for repository access. Do not share anyone's password.

Search, choose, confirm

Step	What to do	Result
1. Search	On the home page, enter From, To or travel date; optionally sort by departure or fare.	Matching future trips, bus types, fares and seat counts appear.
2. Pick seats	Open a trip, inspect the bus-style seat map and choose 1 to 6 available seats.	Selected seats and total fare appear.
3. Hold	Submit the selection while signed in.	Seats are held for 10 minutes; Oracle prevents a double booking.
4. Confirm	Open the held booking and choose the simulated payment/confirmation action.	Status becomes Confirmed and a simulated payment is recorded.
5. Ticket	Open My bookings, select the booking, and print the ticket if desired.	The ticket shows route, times, operator, seats and reference.

What the seat colors mean

- Available seats can be selected. The red/pink treatment highlights availability in the seat map.
- Selected seats are your current choices. Grey seats are unavailable because they are booked, held, or reserved for a live waiting-list offer.

Cancellation and refund

- In My bookings, open your ticket and choose Cancel booking. This cancels the whole booking and releases its seats.
- If the booking had a simulated captured payment, the app records a simulated full refund. No actual payment service or money is involved.

If a hold expires

The 10-minute hold is temporary. Refresh My bookings or the trip seat map. Expired held seats become available again, possibly to a waiting passenger. Start a new hold if you still want seats.

When a trip is full

- Open the full trip and choose Join waiting list while signed in. One active queue entry per passenger and trip is allowed.
- Open My bookings to see your position. The order is first-in, first-out. You can leave the queue if you no longer want the seat.
- When someone cancels, the first waiting passenger gets a seat offer lasting 10 minutes. Accept it from My bookings, then confirm the new held booking. If the offer expires or is declined, the next passenger can receive it.

Staff insights

Staff users open /insights/ to see trip occupancy, simulated captured/refunded/net amounts, and active waiting-list demand. The dashboard reads Oracle reporting views, so a booking, cancellation, or offer changes the displayed figures.

DBMS concepts to show your professor

Concept	Where it appears
Relational design / 3NF	Bus, Seat, Trip, TripSeat, Booking, Payment, WaitlistEntry and audit-event tables
Keys and constraints	Unique seat per bus, one seat inventory per trip, status checks, conditional unique queue/offer indexes
Transactions and locks	All-or-nothing multi-seat holds; row locking prevents simultaneous buyers taking one seat
Triggers	Cross-table seat checks, booking status audit, and same-bus schedule overlap guard
Views and indexes	Occupancy, revenue and FIFO queue views; trip search and queue indexes
Recovery	Oracle Data Pump export and restore drill documented in the repository

For a live SQL Developer walk-through, use docs/live_sql_demo.md and docs/live_sql_demo.sql in the repository. The test commands in README.md verify race handling, rollback, FIFO order, integrity rules and schedule overlap.

06 / TEAM DEMO

A simple three-person run-through

Person	Suggested role	What to show
Leader	Admin and DBMS explainer	Start the server, show buses/trips, Oracle tables and the insights page.
Teammate 1	Passenger A	Sign up, book seats, confirm the simulated payment, show the ticket, then cancel.
Teammate 2	Passenger B	Sign up, join a full trip's queue, inspect the offer and accept it.

Before class

- Check OracleServiceXE and the Oracle listener are running. Start the app and test the home page on the host laptop.
- Test the printed teammate link from another device on the exact Wi-Fi or hotspot you plan to use.
- Confirm you have a future trip for booking and enough seats or a prepared full trip for the waiting-list example.
- Keep the server PowerShell window open. Have the GitHub repository and SQL Developer ready for questions.

Quick fixes

Problem	First check
ORA-01017	Re-enter BUSBOOKING password in the current PowerShell window; check xepdb1.
Site cannot be reached	Is the Django server still running? Is the link current? Are devices on the same network?
College Wi-Fi link fails	Campus device isolation is possible; try a shared phone hotspot and restart the launcher.
Port 8000 in use	Stop the earlier server with Ctrl+C before starting another.
No trips found	Clear filters; check that the trip is future-dated and the bus is active.
PowerShell blocks script	Use the powershell -ExecutionPolicy Bypass -File command on page 3.

Keep improving safely

GitHub saves code and documentation checkpoints. Oracle holds the live users and bookings. Commit code changes locally, then push when internet is available; do not put passwords or database dumps in the repository.